// devlog · entry 03

Why JARs exist, and what Maven fixes

from "too many .class files to send someone" to a build tool that manages my whole dependency tree

PICKS UP FROM
Entry 02 — First Endpoint

FOCUS
JAR · pom.xml · repos · lifecycle

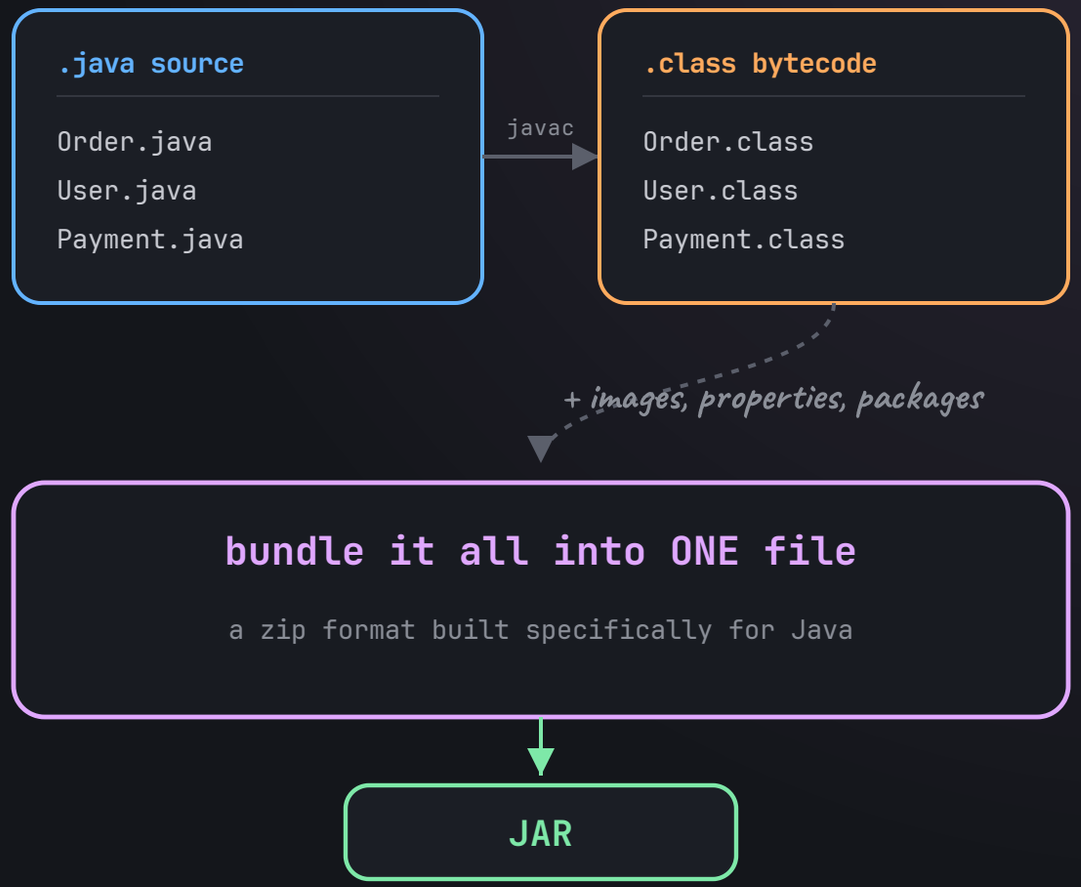
COMMAND LEARNED
mvn clean install

a real project isn't one file

Sharing loose .class files doesn't scale

Every `.java` file compiles into its own `.class` file. A real project has dozens. Handing them to a teammate one by one breaks immediately.

- files get misplaced
- package structure breaks
- resources get left out
- running it becomes guesswork



JAR = Java Archive

Two reasons JAR files matter

01 · to share code easily

Build a reusable calculator library, package it as `calculator.jar`, and any developer can drop it into their project and call my compiled code — nothing loose, nothing missing.

02 · to use external libraries

"I'm using Spring Core" really means my project depends on *their* JARs. Every library I import — MySQL driver, Hibernate, Jackson — is someone else's compiled JAR.

three flavours worth telling apart:

- `library JAR → reusable code, no main()`
- `application JAR → runnable, has main()`
- `Spring Boot "fat JAR" → code + all dependencies inside`

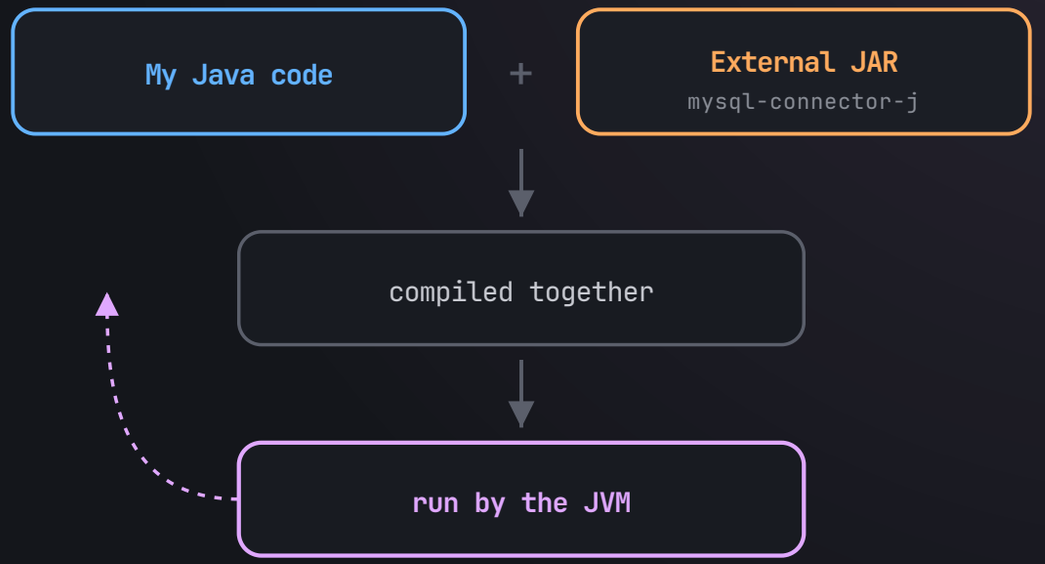
a library JAR ships just my code — a Spring Boot JAR ships my code and everything it depends on, so it runs on its own.

my code + someone else's jar

The classpath is Java's search list

The moment my code calls a class from an external JAR, Java has to answer one question: *"this class isn't in my project — where do I look?"*

classpath tells Java where to search for the classes it needs.



at compile time AND runtime, Java consults the classpath

"not in my project → it's inside that JAR"

manually managing jars falls apart fast

This is exactly where Maven steps in

? where do I download these JARs from

? which version should I use

? what if one JAR depends on another JAR

? what if two teammates use different versions

? what if I forget one required JAR

? what if v1 works but v2 breaks everything

MAVEN SAYS:

"Don't manually download and manage JAR files. Tell me what your project needs — I'll fetch it, manage the versions, and pull in whatever *those* depend on too."

Maven = project management + build automation for Java.

and it isn't a Spring thing — Maven is independent:

Core Java | Spring | Spring Boot | Hibernate | any Java project

pom.xml = Project Object Model

What Maven reads before doing anything

```
<project>
  <groupId>com.myapp</groupId>
  <artifactId>calculator-app</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <dependencies>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>9.6.0</version>
    </dependency>
  </dependencies>
</project>
```

groupId + artifactId + version

"Maven coordinates" — the address that uniquely identifies any project or dependency

packaging

jar · war · pom — what kind of output to build. Defaults to jar.

dependencies

every external library, each addressed by those same 3 coordinates

transitive dependencies

I add A → Maven also pulls B and C that A needs. Dependencies of my dependencies, handled for me.

`1.0.0 → stable` `1.0.0-SNAPSHOT → in development`

the "dependency not found" moment

Local repo and Maven Central

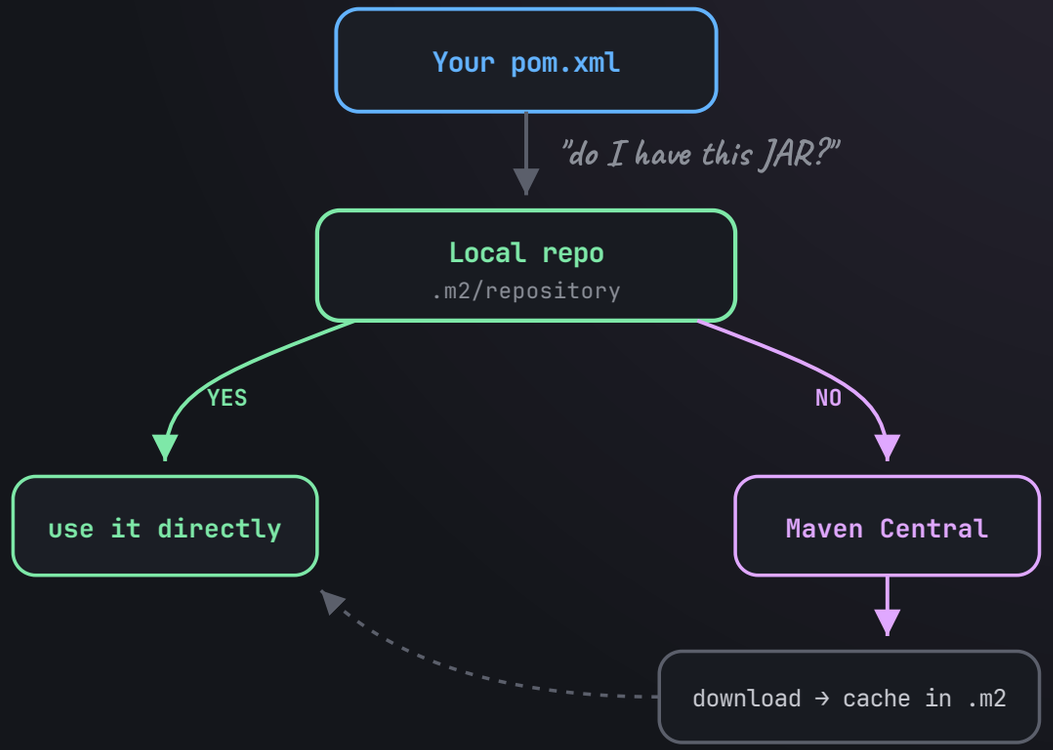
LOCAL REPOSITORY

A folder on my own machine holding every dependency Maven has downloaded.

`C:\Users\<username>\.m2\repository`

MAVEN CENTRAL

The huge public library of Java packages — MySQL, Spring, JUnit, Jackson, Lombok, Hibernate.



first build is slow — every build after is cached and fast

the most important rule in maven

Running a phase runs every phase before it

Maven never skips ahead — earlier phases always run first.

there are three lifecycles in total:

`clean → wipes the target/ folder`

`default → validate through deploy (above)`

`site → generates project docs`

two lifecycles, chained

mvn clean install, decoded

Maven runs the **clean** lifecycle first — deleting `target/` — then the **default** lifecycle all the way through `install`.

a fully fresh, tested build — installed locally so my other projects can depend on it.

entry 03, wrapped

Maven exists so I never manage JARs by hand

- ✓ .java compiles to .class, the JVM runs .class
- ✓ a JAR bundles compiled code + resources into one file
- ✓ classpath = where Java looks for required classes
- ✓ tracking JAR versions by hand doesn't scale
- ✓ pom.xml + groupId/artifactId/version = Maven's map
- ✓ local .m2 caches, Maven Central supplies
- ✓ a lifecycle phase always runs everything before it
- ✓ mvn clean install = fresh, tested, installed build

Maven exists because real Java projects need a standard way to manage structure, dependencies, builds, packaging, and sharing.

next up: Servlets · Tomcat · Spring Core · IoC · Dependency Injection · Beans